vaadin

Thinking in signals

Leif Åstrand
Thinkerer

Agenda

Sections

01 How to think

02 How it works

03 Q/A

vaadin

01

How to think

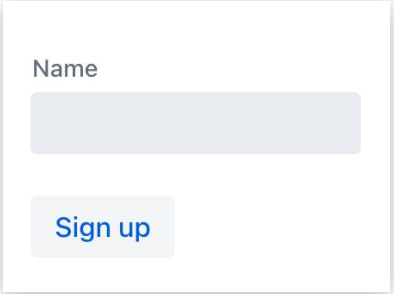
A great form
A greater form
A loop

vaadin

IT'S A GREAT FORM

A simple sign-up form

```
add(new TextField("Name"));
add(new Button("Sign up"));
```



vaadin

STILL REASONABLY SIMPLE

Add validation

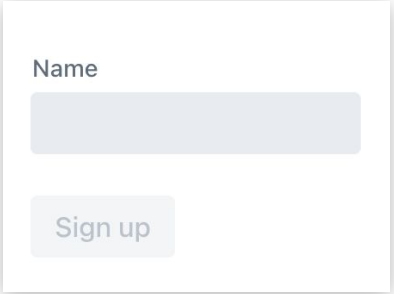
```
add(new TextField("Name", event -> {
  submit.setEnabled(!event.getValue().isBlank());
}));
```

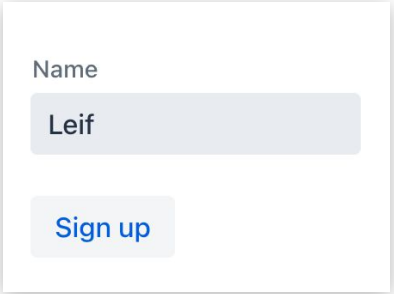
vaadin

STILL REASONABLY SIMPLE

Add validation

```
var submit = new Button("Sign up");
submit.setEnabled(false);
add(new TextField("Name", event -> {
  submit.setEnabled(!event.getValue().isBlank());
}));
add(submit);
```



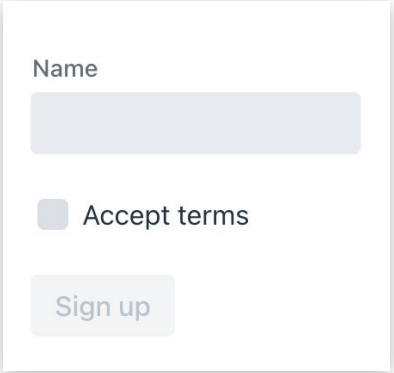


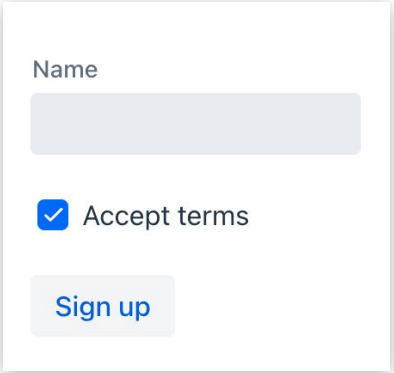
vaadin

HELLO, BUGS!

Add terms of service

```
var submit = new Button("Sign up");
submit.setEnabled(false);
add(new TextField("Name", event -> {
  submit.setEnabled(!event.getValue().isBlank());
}));
add(new Checkbox("Accept terms", event -> {
  submit.setEnabled(event.getValue() == true);
}));
add(submit);
```





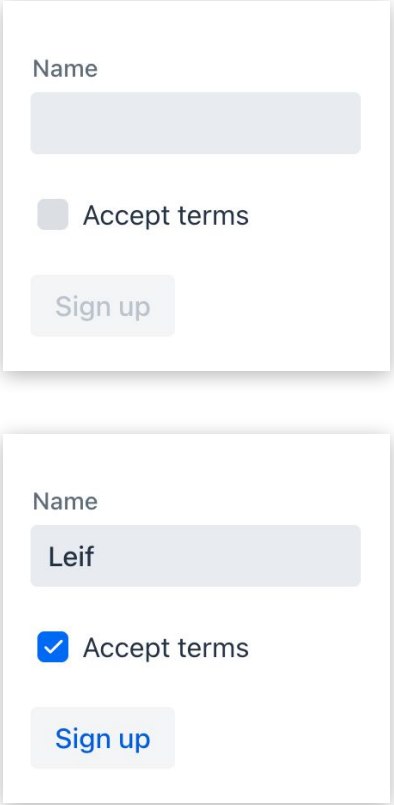
vaadin

TAMING COMPLEXITY MANUALLY

Apply changes consistently

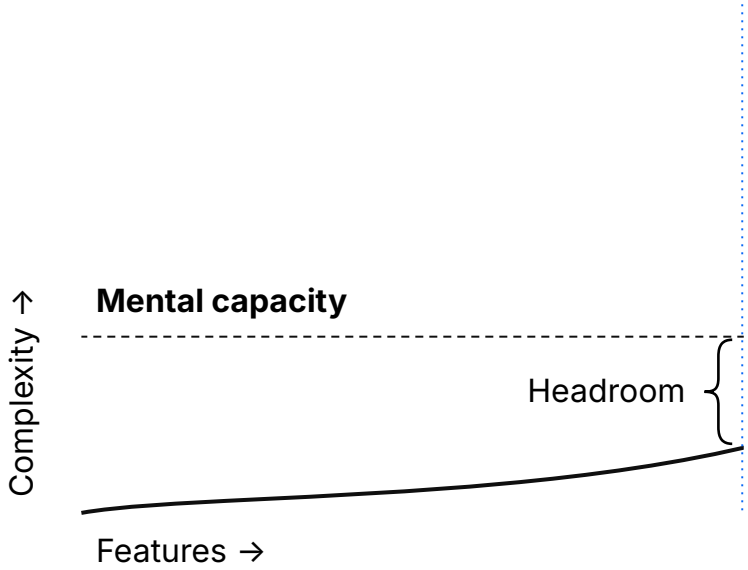
```
private void updateSubmit() {
  submit.setEnabled(
    !name.getValue().isBlank()
      && terms.getValue() == true
  );
}

// + introduce instance fields
// + call updateSubmit() in all the right places
```



vaadin

Exponential complexity



SIGNALS!

Make it a framework feature

```
var name = new ValueSignal<>("");


add(new TextField("Name", name));


var submit = new Button("Sign up");
submit.bindEnabled(() -> !name.value().isBlank());

add(submit);
```

vaadin

SIGNALS!

Make it a framework feature

```
var name = new ValueSignal<>("");
var accept = new ValueSignal<>(false);

add(new TextField("Name", name));
add(new Checkbox("Accept terms", accept));

var submit = new Button("Sign up");
submit.bindEnabled(() -> !name.value().isBlank()
  && accept.value() == true);
add(submit);
```

vaadin

ALMOST LIKE A JAVA-BASED TEMPLATE LANGUAGE

No components in variables

```
var name = new ValueSignal<>("");
var accept = new ValueSignal<>(false);

add(new TextField("Name", name));
add(new Checkbox("Accept terms", accept));

add(new Button("Sign up") {{
  bindEnabled(() -> !name.value().isBlank()
    && accept.value() == true);
}});
```

vaadin

RULE 1

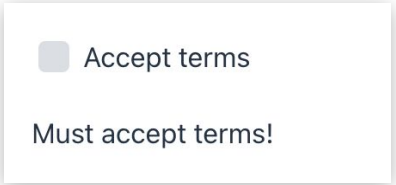
Don't touch components in event listeners

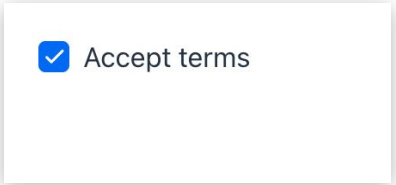
vaadin

OBVIOUS BUT MESSY

Conditional rendering

```
var warning = new Span("Must accept terms!");
add(new Checkbox("Accept terms", event -> {
  if (event.getValue() == false) {
    add(warning);
  } else {
    remove(warning);
  }
}));
add(warning);
```



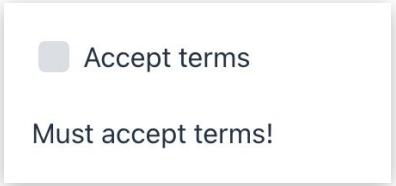


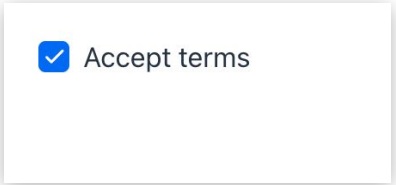
vaadin

THE DECLARATIVE WAY

Bind visibility

```
var accept = new ValueSignal<>(false);
add(new Checkbox("Accept terms", accept));
add(new Span("Must accept terms") {{
  bindVisible(() -> accept.value() == false);
}});
```





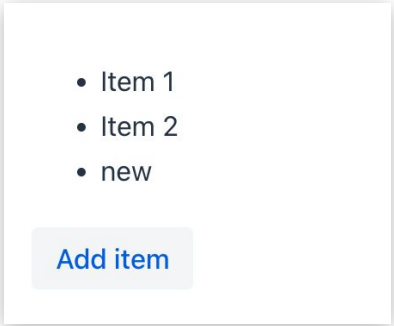
vaadin

MANUAL STATE REPLICATION

Dynamic children

```
List<Item> items = ...;
var ul = new UnorderedList();
items.forEach(i -> ul.add(new ListItem(i.text())));

add(ul);
add(new Button("Add item", click -> {
  var item = new Item("new");
  items.add(item);
  ul.add(new ListItem(item.text()));
}));
```



vaadin

RULE 2

Initialize and update in the same way

vaadin }>

JUST DEFINE WHAT YOU WANT

Dynamic children

```
ListSignal<Item> items = ...;
add(new UnorderedList(items, signal -> new ListItem(
  signal.map(Item::text)
)));
add(new Button("Add item", click -> {
  items.insertLast(new Item("new"));
}));
```

vaadin

UPDATES AND REMOVALS JUST WORK™

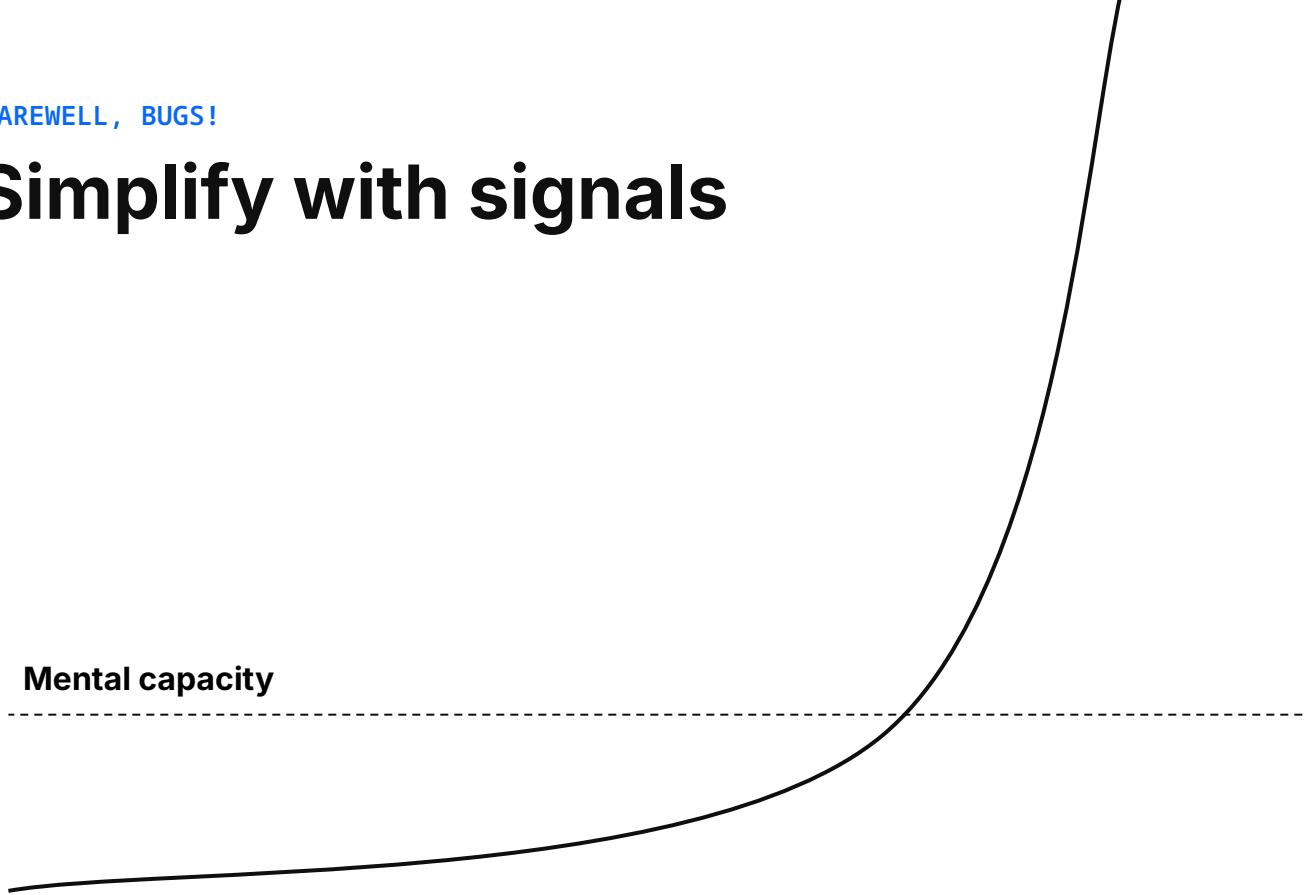
Dynamic children

```
ListSignal<Item> items = ...;
add(new UnorderedList(items, signal -> new ListItem(
  new Text(signal.map(Item::text)),
  new Button("Update", e -> signal.value(new Item("update"))),
  new Button("Remove", e -> items.remove(signal))
)));
add(new Button("Add item"), click -> {
  items.insertLast(new Item("new"));
});
```

vaadin

FAREWELL, BUGS!

Simplify with signals



ONE RULE TO RULE THEM ALL

Make authoritative state explicit
Derive everything else from it

vaadin }>

02

How it works

- Magic listeners
- Magic cleanup
- Magic locking
- Magic fan-out

vaadin

A LOOK AT THE PLUMBING

Effect = automatic signal change listener

```
Signal<String> signal = new ValueSignal<>("value");

// prints "value"
Signal.effect(() -> System.out.println(signal.value()));

// prints "update"
signal.value("update");
```

vaadin

A DEEPER LOOK AT THE PLUMBING

Effect = automatic signal change listener

```
static ThreadLocal<UsageTracker> current;

private void runEffect() {
  var tracker = new UsageTracker();
  current.set(tracker);
  effectCallback.run();
  current.remove();
  tracker.usages().forEach(signal ->
    signal.onChange(this::invalidate));
}
```

```
public T value() {
  current.get().register(this);
  return this.value;
}
```

vaadin

A LOOK AT THE PLUMBING

Signal support in components

```
public void bindText(Signal<String> signal) {

}
```

vaadin

A LOOK AT THE PLUMBING

Components delegate to Element

```
public void bindText(Signal<String> signal) {
  getElement().bindText(signal);
}
```

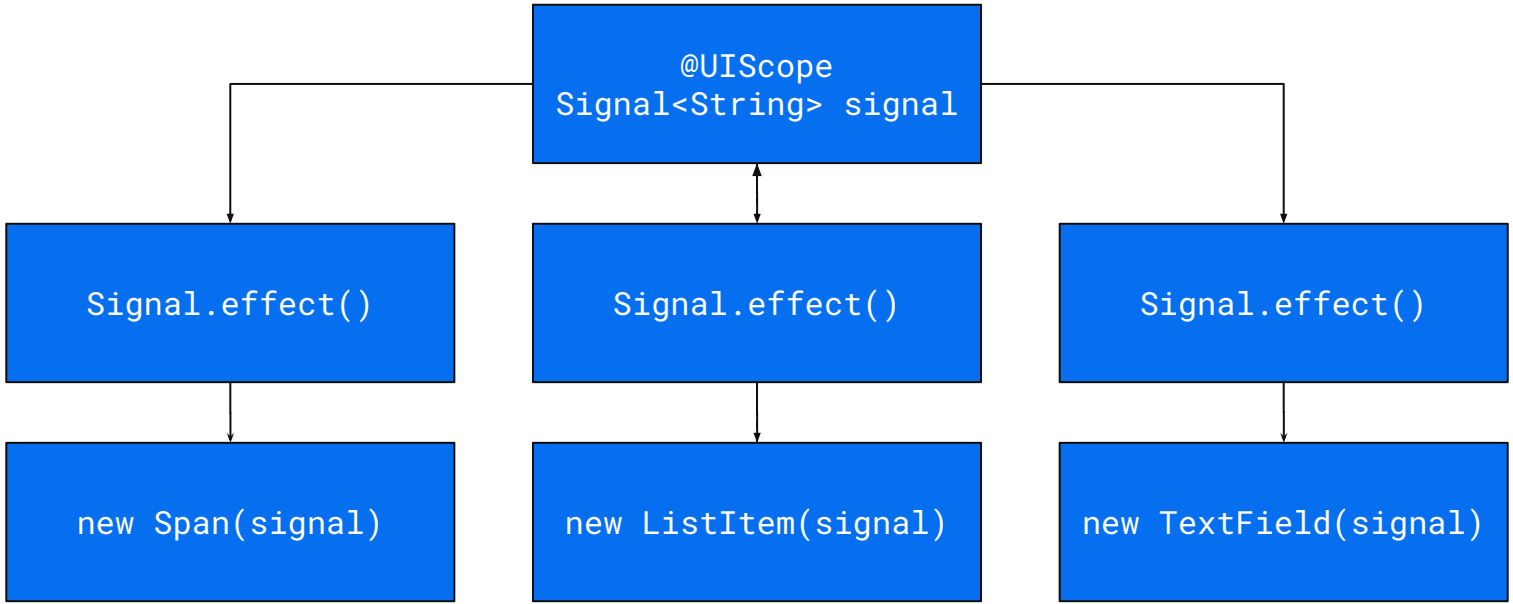
vaadin

A LOOK AT THE PLUMBING

It's an effect behind the scenes

```
public void bindText(Signal<String> signal) {
  Signal.effect(() -> this.setText(signal.value()));
}
```

References everywhere



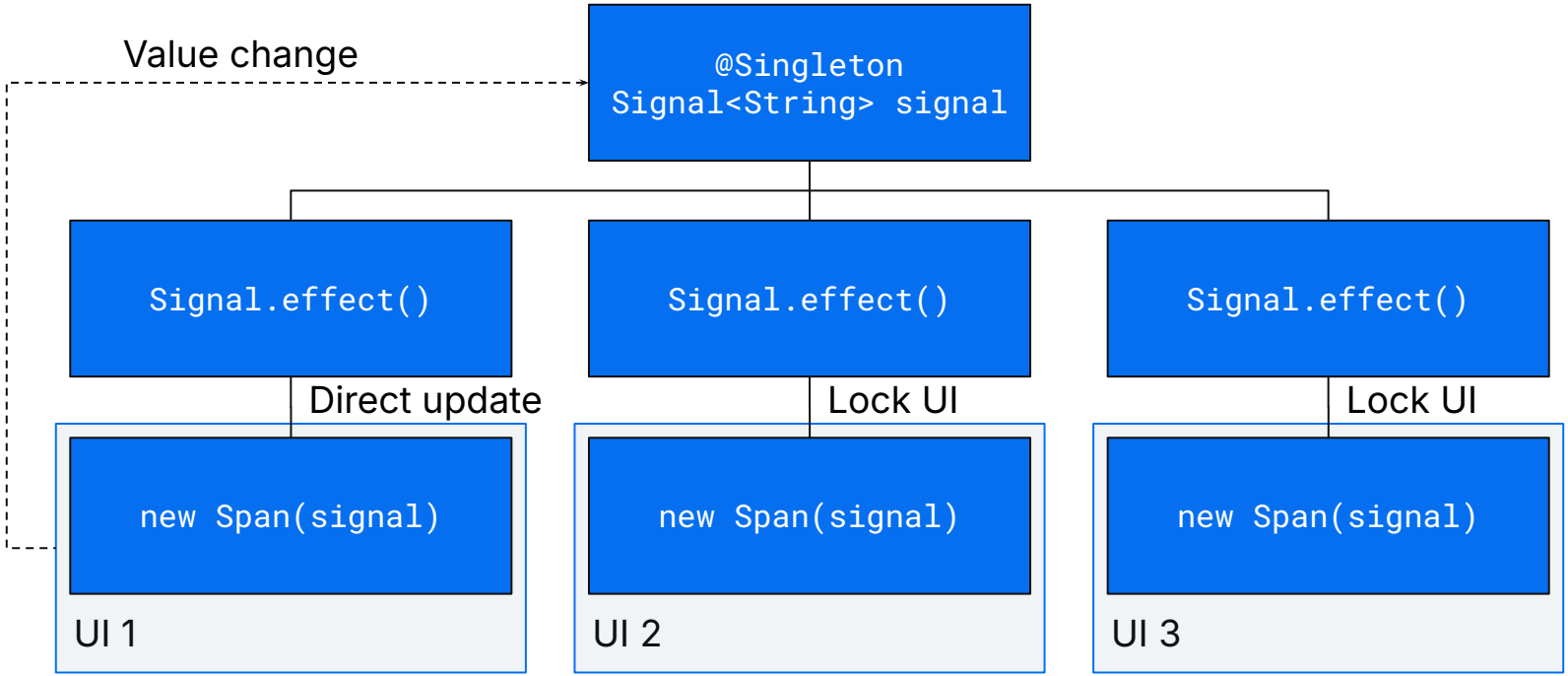
A LOOK AT THE PLUMBING

Listen only while attached

```
public void bindText(Signal<String> signal) {
  addAttachListener(attach -> {
    var cleanup = Signal.effect(() -> setText(signal.value()));
    addDetachListener(detach -> {
      cleanup.run();
      detach.unregisterListener();
    });
  });
}
```

A LOOK AT THE PLUMBING

Shared between UIs



vaadin

A LOOK AT THE PLUMBING

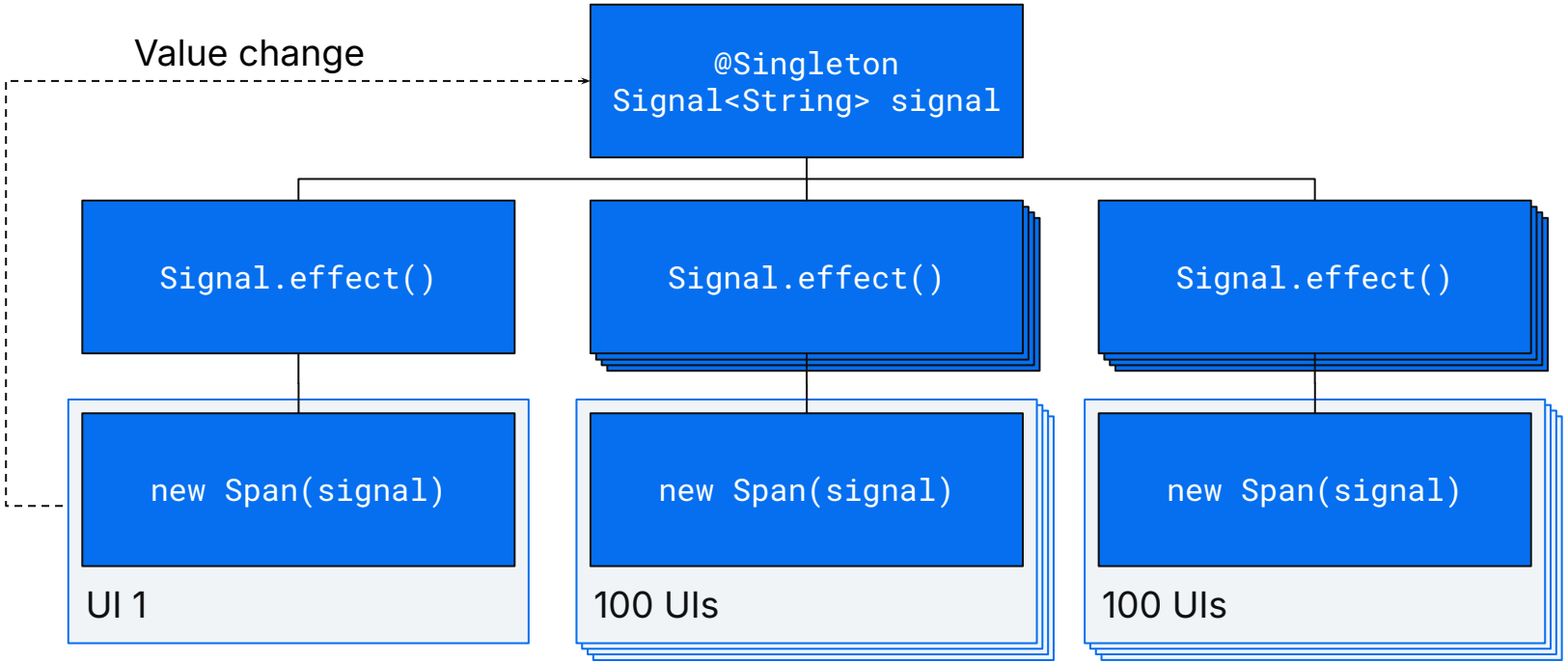
Lock session when necessary

```
public void bindText(Signal<String> signal) {
  addAttachListener(attach -> {
    var cleanup = Signal.effect(() -> {
      if (UI.getCurrent() == attach.getUI()) {
        setText(signal.value());
      } else {
        attach.getUI().access(() -> {
          setText(signal.value());
        });
      }
    });
```

vaadin

A LOOK AT THE PLUMBING

Updating many UIs



A LOOK AT THE PLUMBING

Avoid blocking the initiator

```
public void bindText(Signal<String> signal) {
  addAttachListener(attach -> {
    var cleanup = Signal.effect(() -> {
      if (UI.getCurrent() == attach.getUI()) {
        setText(signal.value());
      } else {
        runInBackground(() -> attach.getUI().access(() -> {
          setText(signal.value());
        }));
      }
    });
```

vaadin

A LOOK AT THE PLUMBING

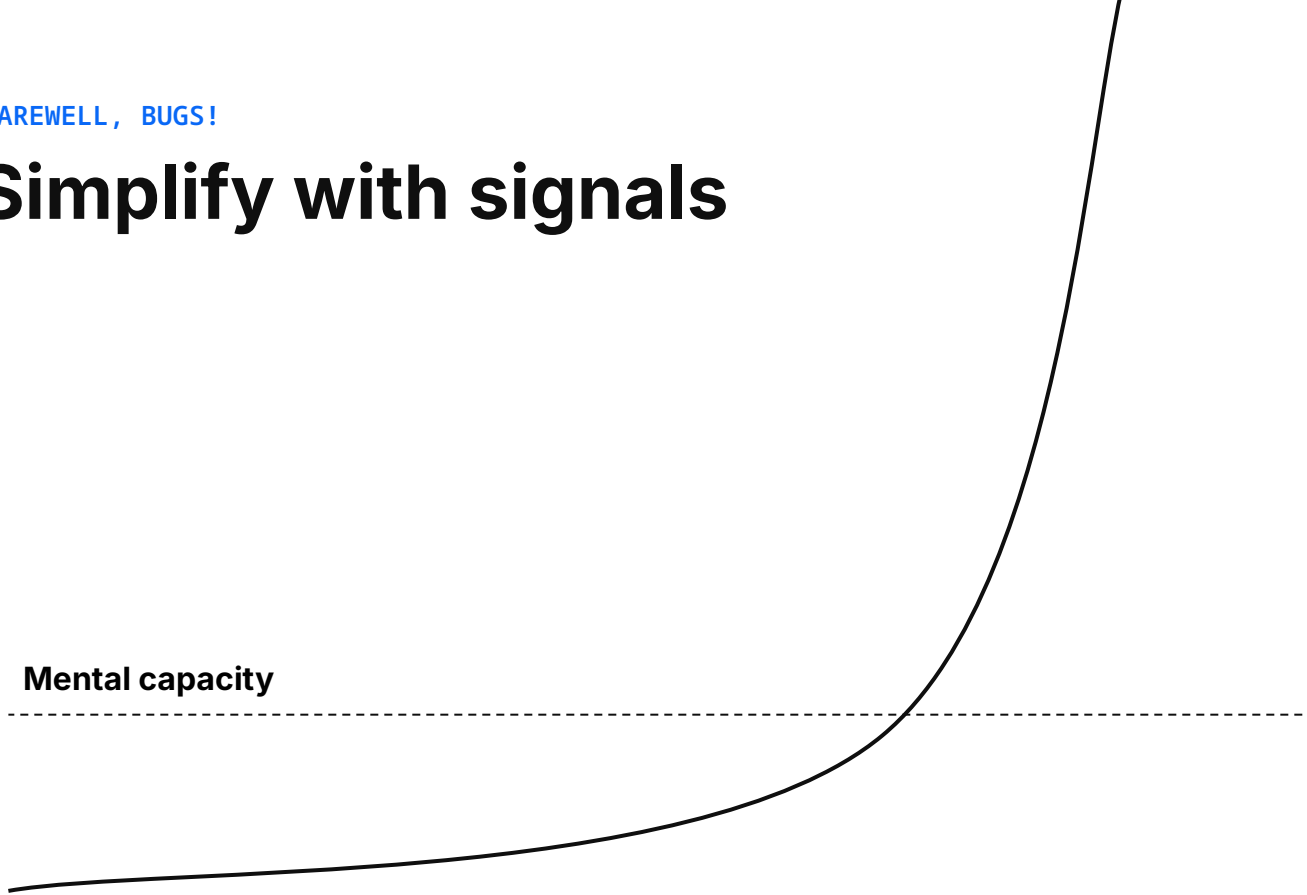
Extract the pattern

```
public void bindText(Signal<String> signal) {
  ComponentEffect.effect(this, () -> setText(signal.value()));
}
```

vaadin

FAREWELL, BUGS!

Simplify with signals



THINKING IN SIGNALS

Make authoritative state **explicit**
Derive everything else from it

Don't touch components in event listeners

Automatic listeners

Automatic cleanup

Initialize and update in the same way

Automatic locking

Automatic fan-out

vaadin

FAQ

Roadmap

Preview in 24.8

Core signal types
ComponentEffect

Preview in 25.0

Element APIs
Component APIs?

25.1+

Out of preview?

Also under consideration

Debugging
Binder integration
Router integration
Lazy loading
Clustering

Q/A

vaadin

Thank you!

vaadin